

YEREL · GİZLİLİK-KORUMALI · YAPAY ZEKÂ

Şirket-Sorgu

Türkçe Soru → SQL → Otomatik Rapor

Şirket verisini **hiç dışarı çıkarmadan**, tamamen yerel çalışan bir yapay zekâ ajanı: çalışan Türkçe sorar, sistem güvenli SQL üretir, çalıştırır ve doğrulanmış bir Türkçe rapor döndürür.

Hazırlayan: Harun Gökçe
Tarih: 28 Haziran 2026
Sürüm: MVP (Faz 1-2)

Bağlantılar

GitHub: [«repo-linkini-buraya-ekle»](#)
Demo videosu: [«video-linkini-buraya-ekle»](#)
LinkedIn: [«profil-linkini-buraya-ekle»](#)

İçindekiler

İçindekiler	2
1. Yönetici Özeti	3
2. Problem: Text-to-SQL Neden Zor, Gizlilik Neden Önemli?	4
3. Bu Projeyi Ne Ayırıştırıyor?	5
4. Mimari: Uçtan Uca Akış	6
5. Kontrol Katmanları: Sistemin Kalbi	8
6. Halüsinasyonsuz Rapor	9
7. Veritabanı: Neden PostgreSQL?	10
8. Model Seçimi: Hangisi Ne Kadar Yeterli?	11
9. Yerel (Local) Çalışmanın Avantajları	12
10. Neye Dikkat Ettik (Dersler & Tuzaklar)	13
11. Nasıl Kurulur ve Çalıştırılır	14
12. Doğruluk Kanıtı ve Test	15
13. Yol Haritası	16
14. Kapanış	17

1. Yönetici Özeti

Bu proje, bir şirketin kendi veritabanına **doğal Türkçe ile** soru sorabilmesini sağlayan, **tamamen yerel (local) çalışan** bir text-to-SQL yapay zekâ ajanıdır. Kullanıcı “en çok ciro yapan 5 ürün hangisi?” diye sorar; sistem soruyu güvenli bir SQL sorgusuna çevirir, salt-okunur çalıştırır, sonucu işler ve **doğrulanmış** bir Türkçe rapor döndürür.

Tek cümlede değer önermesi

Kurumsal text-to-SQL hâlâ çözülmemiş zor bir problem (Spider 2.0 benchmark’ında GPT-4o bile yalnızca ~%10). Bu projede soruna **mühendislik disipliniyle** ve **veriyi hiç dışarı çıkarmadan** yaklaşıldı.

Kimin için?

- **İşletmeler:** veri analizi için SQL bilmeyen ekiplere self-servis.
- **Mühendisler:** katmanlı kontrol + RAG few-shot mimarisi referansı.
- **Gizlilik-hassas kurumlar:** veri makineden hiç çıkmaz (KVKK/GDPR dostu).

Ölçülen sonuç (3B model, dizüstü)

- Temel/orta sorular: **%87–95 doğruluk**
- İleri analitik (pencere fn., percentile): **~%20**
- Güvenlik testi: **15/15** · Türetilmiş metrik: **3/3**
- Tüm bu sayılar gerçekten ölçüldü — uydurma yok.

Projenin ayırt edici yanı tek bir model değil, **mimaridir**: şema linkleme, RAG few-shot örnekleme, çok-katmanlı güvenlik/doğruluk kontrolü, türetilmiş-metrik denetimi ve “sayıyı kod hesaplar, LLM yalnızca yorumlar” ilkesiyle **halüsinasyonsuz** rapor.

2. Problem: Text-to-SQL Neden Zor, Gizlilik Neden Önemli?

Zorluk gerçek

Basit akademik testlerde (Spider 1.0) en iyi sistemler ~%86–91 başarıya ulaşır. Ama **gerçek kurumsal şemalarda** (Spider 2.0 — 1000+ kolonlu veritabanları, 100+ satırlık SQL) tablo tersine döner:

Benchmark / Sistem	Doğruluk
Spider 1.0 (basit, akademik)	~%86–91
BIRD (gerçekçi, gürültülü)	en iyi %81.95 / insan %92.96
Spider 2.0 — GPT-4o (gerçek kurumsal)	~%10
Spider 2.0 — o1-preview	~%21

Çıkarım

Model seçimi tek başına belirleyici **değil**. Doğruluğu **mimari** belirler — retrieval, şema linkleme, kontrol katmanları, semantik denetim. Bu projenin neden “tek model + tek atış” değil de katmanlı bir pipeline olarak tasarlandığının birinci-elden gerekçesi budur.

Gizlilik: şirketlerin asıl korkusu

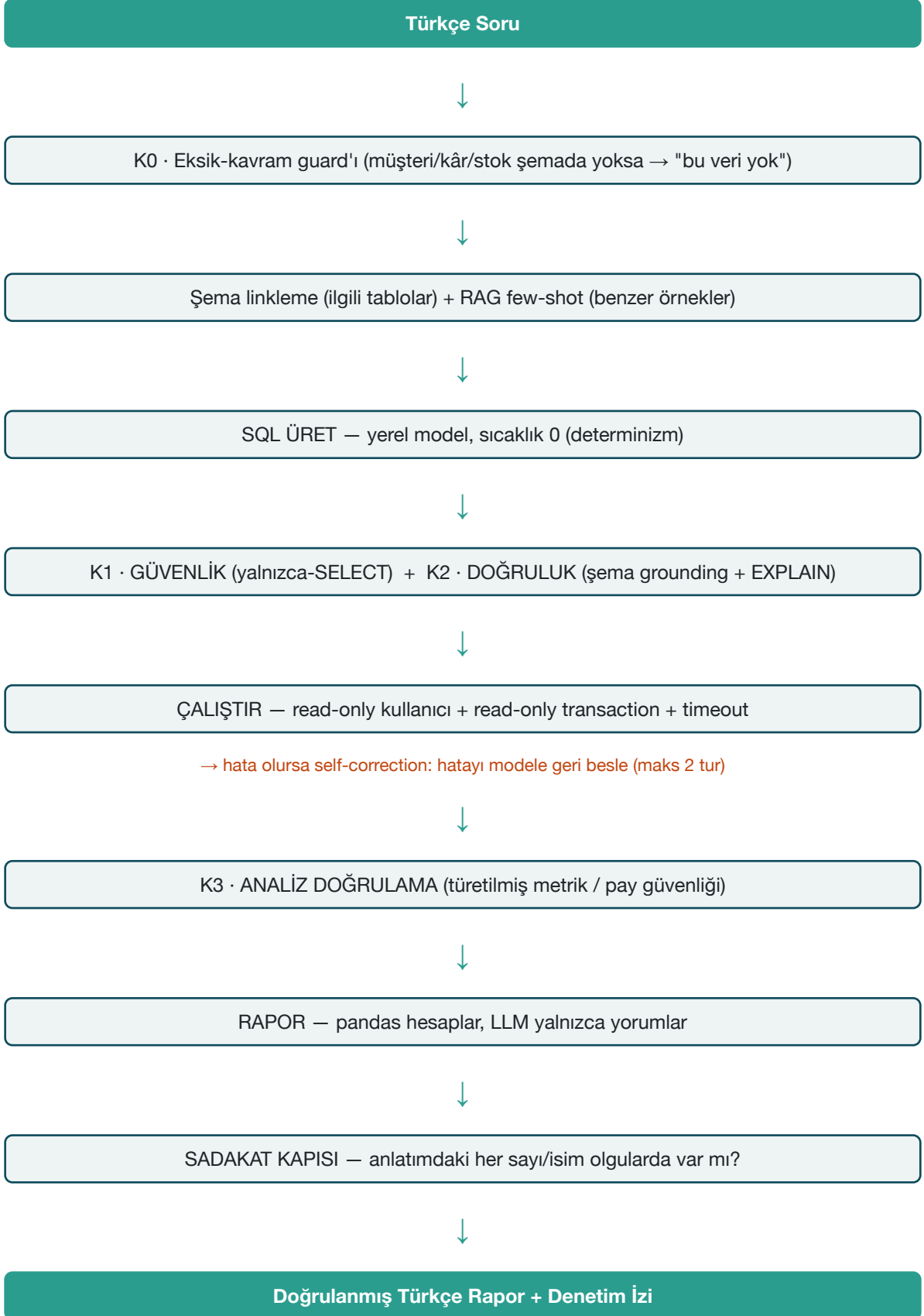
Çoğu hazır çözüm soruyu ve şemayı bir bulut API’sine (OpenAI, vb.) gönderir — yani **veri şirketten çıkar**. Birçok kurum için bu pazarlıksız bir engeldir. Bu proje baştan sona **yerel modelle** çalışır: soru, şema ve veri bilgisayardan/sunucudan **hiç dışarı çıkmaz**.

3. Bu Projeyi Ne Ayırıştırıyor?

Sıradan demo	Bu proje
Bulut API → veri dışarı çıkar	%100 yerel — gizlilik-korumalı
SQL'i körü körüne çalıştırır	Çok-katmanlı güvenlik (read-only kullanıcı + read-only transaction + AST denetimi + timeout)
"Çalışıyor" der, kanıt yok	Dürüst doğruluk ölçümü (kendi gold set + execution accuracy)
Tek model, tek atış	Araştırma-temelli mimari (şema linkleme + few-shot + self-correction)
Sayı LLM uydurur	Sayı pandas hesaplar , LLM yalnızca yorumlar → halüsinasyon yok

4. Mimari: Uçtan Uca Akış

Sistem bir soruyu, her adımı denetlenen bir hat üzerinden cevaba dönüştürür:



Modüler tasarım

Her katman tek sorumluluk taşır ve ayrı dosyadadır: `sql_guvenlik.py` (güvenlik), `sql_kontrol.py` (doğruluk), `analiz_kontrol.py` (metrik denetimi), `sema.py` (şema), `retrieval.py` (few-shot/RAG), `rapor.py` (rapor), `db_sorgu.py` (read-only DB), `runner.py` (orkestratör).

5. Kontrol Katmanları: Sistemin Kalbi

Tasarımın merkezinde bir ilke var: **her adıma kontrol koymak aşırı mühendisliktir**. Bunun yerine “hata olasılığı $\times$ etki” en yüksek **kritik noktalara** odaklanıldı. Ve sıralama bilinçli: **önce ucuz deterministik kontroller, en son pahalı LLM**.

Kontrol	Ne yakalar	Tür	Maliyet
K0	Şemada karşılığı olmayan kavram (müşteri, kâr, stok) → uydurma yerine “bu veri yok”	deterministik	~0
K1	Yazma/silme/DDL/injection — yalnızca SELECT’e izin	deterministik	~0
K2	Uydurma tablo/kolon (AST grounding) + EXPLAIN ile tip/şema garantisi	deterministik	~0
K3	Türetilmiş metrik hatası (kesik sonuçta yanlış “pay/yüzde”)	deterministik	~1 ek sorgu
Sadakat	Anlatımda olgularda olmayan sayı/isim → reddet, deterministik özete düş	deterministik	~0

Neden bu noktalar?

- **K1 (güvenlik)**: etki felaket (silme/sızma), olasılık düşük → pazarlıksız kontrol.
- **K2 (doğruluk)**: uydurma kolon en yaygın LLM hatası; EXPLAIN , çalıştırmadan tip/şema uyumunu %100 garanti eder — üstelik bedava. **En yüksek getirili nokta.**
- **K3 (anlam)**: “çalışır ama yanlış cevap” yalnızca burada yakalanır; sadece risk sinyalinde (kesik sonuç) tetiklenir.

Tasarım felsefesi

Kendinden emin yanlış cevap vermektense “emin değilim / bu veri yok” demek daha sağlamdır. Koşul sağlanmazsa sistem metriği bastırır ve çekince ekler. Dürüstlük, hem doğruluk hem de güven kazandırır.

6. Halüsinasyonsuz Rapor

Rapor katmanı **hibrit** tasarlandı çünkü “LLM hesaplasın” güvenilirmez (akademik testlerde sayısal doğruluk ~%78):

1. `ozetle()`
DETERMİNİSTİK
pandas hesaplar

2. `icgoru_sec()`
KURAL TABANLI
en önemli bulgular

3. `anlat()`
LLM YALNIZCA DİL
Türkçe yorum

Tüm sayılar (toplam, ortalama, pay, sıralama) **kod** tarafından hesaplanır; LLM yalnızca akıcı Türkçe cümleyi yazar ve “verilen sayı dışında rakam üretme” talimatı alır. Üstüne **sadakat kapısı**: anlatımdaki her sayı/isim yapılandırılmış olgularda yoksa, anlatım reddedilip %100 sadık deterministik özete düşülür.

Gerçek çıktı — "en çok ciro yapan 5 ürün hangisi?"

ÜRETİLEN SQL:

```
SELECT u.ad AS urun, SUM(s.toplam_tutar) AS ciro
FROM satislar s JOIN urunler u ON u.id = s.urun_id
GROUP BY u.ad ORDER BY ciro DESC LIMIT 5
```

SONUÇ: Bal 460g 175.680 · Kaşar 400g 167.475 · Peynir 500g 132.820 ...

ÖZET : Bal 460g, tüm ciro (1.646.768) içinde %10.7 pay tutuyor.

İZ : K0 ✓ K1 ✓ K2 ✓ Çalıştırma ✓ K3: gerçek toplam ayrı çekildi Sadakat ✓

Not: “pay” hesabı için payda (1.646.768) ayrı, güvenli bir sorguyla çekildi — LIMIT’li sonucun toplamı **değil**. Bu, K3’ün asıl işidir.

7. Veritabanı: Neden PostgreSQL?

PostgreSQL, gizlilik-korumalı ve güvenli bir text-to-SQL hattı için ideal özelliklere sahip:

Özellik	Bu projeye katkısı
Salt-okunur kullanıcı (GRANT)	Yazma/silme'yi fiziksel olarak imkânsız kılar — en güçlü koruma katmanı
Read-only transaction	Çalıştırma sırasında ikinci savunma hattı
EXPLAIN (yan etkisiz)	Sorguyu çalıştırmadan şema/tip uyumunu %100 doğrular (K2)
<code>information_schema</code>	Şemayı runtime'da okuma → kod içine gömülü DDL yok, şema değişince kırılmaz
Olgunluk & yaygınlık	Kurumlarda zaten standart; ek lisans/maliyet yok

Hangi diğer veritabanlarıyla yapılabilir?

Mimari veritabanı-agnostiktir; SQL diyalekti değişse de yaklaşım aynıdır:

Veritabanı	Not
MySQL / MariaDB	Çok yaygın; read-only kullanıcı + EXPLAIN mevcut, diyalekt farkları küçük
SQLite	Tek-dosya, kurulumuz; küçük/yerel demolar ve gömülü kullanım için ideal
DuckDB	Analitik (OLAP) için çok hızlı; CSV/Parquet üzerinde doğrudan SQL
SQL Server / Oracle	Kurumsal; diyalekt ve izin modeli farklı ama aynı kontrol felsefesi geçerli

Diyalekt notu

Tek değişen, üretim prompt'undaki SQL diyalekti ve EXPLAIN söz dizimidir. Kontrol katmanları, rapor ve güvenlik mantığı **aynen** taşınır. Bu projede PostgreSQL seçildi çünkü güçlü izin modeli + EXPLAIN + kurumsal yaygınlık üçlüsünü en temiz sunan veritabanı.

8. Model Seçimi: Hangisi Ne Kadar Yeterli?

Donanım kısıtı belirleyici oldu: geliştirme makinesi **MacBook M1, 8 GB RAM**.

Model	Boyut	8GB M1'de?	Rol
qwen2.5-coder 3B	1.9 GB	✓ rahat	Yerel geliştirme + test (ana model)
qwen2.5-coder 7B	4.7 GB	✗ çökertti	Yalnızca kiralık GPU'da
Qwen2.5-Coder 32B	~18–24 GB	✗ imkânsız	Kiralık GPU'da final/ileri analitik

Sahadan ders: 7B, 8GB makineyi çökertti

7B modeli (4.7 GB) yerel çalıştırınca, model + PostgreSQL + Python + embedding toplamı 8 GB'ı aştı → swap → makine kilitlendi. Karar: **geliştirme/test hep 3B yerelde; 7B/32B yalnızca kiralık GPU'da**. Geçişte yalnızca iki ortam değişkeni (YEREL_MODEL , VLLM_BASE_URL) değişir; kod aynı kalır.

3B'nin doğruluk haritası (ölçülen)

Doğru cevabı önceden bilinen sorularla iki stres turu çalıştırıldı:

Soru sınıfı	Önce	İyileştirme sonrası
Temel/orta (agregasyon, filtre, 2–3 join, oran, guard)	11/20	19/20 (%95)
Orijinal gold set	6/8	7/8 (%87.5)
Türetilmiş metrik (K3)	3/3	3/3
İleri analitik (window, top-N-per-group, percentile, nested)	—	3/15 (%20)

3B'nin kalıcı duvarı

3B; **pencere fonksiyonu (ROW_NUMBER), grup-başına-top-N, çok-seviyeli iç içe agregasyon ve percentile/medyan** sorularını çözemiyor — deposuna doğru örnek konsa bile genellestemiyor. Bu, few-shot ile aşılamayan **kavramsal bir sınır**. Bu tür ileri analitik için 7B/32B (GPU) gerekir.

Net çıkarım

3B modeli, **1.9 GB ile bir dizüstüde**, temel ve orta zorluktaki iş sorularında %87–95 doğrulukla çalışır — gündelik raporlamanın büyük kısmı budur. İleri analitik gerektiğinde aynı kod, kiralık GPU'da 7B/32B'ye geçer. Bu, “küçüklerle başla, gerektiğinde büyüt” stratejisidir.

9. Yerel (Local) Çalışmanın Avantajları

Gizlilik & uyum

Veri, şema ve sorular makineden **hiç çıkmaz**. KVKK/GDPR hassas kurumlar için pazarlıksız avantaj. Bulut API'lerinde veri üçüncü tarafa gider; burada gitmez.

Sıfır API maliyeti

Token başına ücret yok. Sorgu sayısı arttıkça maliyet **artmaz**. Yalnızca bir kez donanım/elektrik. Bulut API'de her sorgu para demektir.

Bağımsızlık

İnternet kesintisi, API kotası, fiyat değişikliği, model emekliye ayrılması **etkilemez**. Sistem tamamen senin kontrolünde.

Öngörülebilirlik

Sabit donanımda **tutarlı** gecikme; dış servis yavaşlaması yok. Sıcaklık 0 ile demo determinizmi.

Ne zaman GPU'ya geçmeli?

Yerel 3B gündelik raporlama için yeter. **İleri analitik (pencere fonksiyonu, karmaşık çok-tablolu analiz) veya final/canlı demo gerektiğinde**, aynı kod kiralık bir GPU'da (ör. RunPod A6000, ~\$0.49/saat) 7B/32B ile çalıştırılır — yine tek-kiracı, veri yine dışarı çıkmadan. Sadece ihtiyaç anında aç, sonra kapat.

10. Neye Dikkat Ettik (Dersler & Tuzaklar)

Bu bölüm, geliştirme sırasında **acıyla** öğrenilen ve sağlamlığı belirleyen noktaları içerir:

Tuzak	Ders / Çözüm
7B çökmesi	8GB'de 7B çalıştırma; yerel = 3B, ağır = GPU. Fallback'ı 3B yap ki env unutulsa çökmesin.
Türkçe locale	DB'de <code>lower('Ç')='ç'</code> (katlamıyor) → <code>ILIKE/~*</code> çöküyor. Çözüm: <code>lower(unaccent(col))</code> (unaccent önce). “kola” ≠ “çikolata”, “çay” = “Çay”.
Decimal bug	PostgreSQL NUMERIC → pandas onu metin sanıyor + JSON bozuyor. float'a çevir.
Auto-learning zehiri	Çalışan-ama-yanlış SQL'i few-shot'a kaydetmek modeli kirlendirir. Otomatik öğrenmeyi kapat ; yalnızca doğrulanmış örneklerle besle.
Pay non-additive	LIMIT'li sonuçta “toplam içinde %” yanlış payda alır. Gerçek toplamı ayrı çek; çekemezsen bastır + çekince (K3).
Değer linkleme	Model “Temizlik”in kategori değeri olduğunu bilmiyordu → şemaya gerçek değerleri enjekte et (kategori/şehir/marka).
Guard Türkçe-ek tuzağı	<code>\bmüşteri\b</code> “müşterimiz”i geçirir. Kelime-başı + ek eşleşmesine geç; çakışan kısa kökleri (kar) temizle.
3B'de prompt şişmesi	Küçük modele fazla talimat (ör. büyük terim sözlüğü) geri teper — başka soruları bozar. Az ama öz prompt.
Execution-match kırılganlığı	Kolon adı/sayısı farkı doğru cevabı yanlış sayabilir. Değer-temelli karşılaştır; kendi gold set'ini çift kontrol et.

11. Nasıl Kurulur ve Çalıştırılır

Gereksinimler

- Python 3.11+ · PostgreSQL · [Ollama](#) (yerel model çalıştırıcı)
- ~2 GB boş disk (3B model) + ~2 GB RAM serbest

Adım adım

```
# 1) Repoyu klonla
git clone <repo-linki> && cd Local-sirket-sorgu

# 2) Sanal ortam + bağımlılıklar
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt

# 3) Yerel modeli indir (Ollama açık olmalı)
ollama pull qwen2.5-coder:3b

# 4) Demo veritabanını kur
createdb sirket_demo
export DATABASE_URL="postgresql://<kullanici>@localhost:5432/sirket_demo"
.venv/bin/python demo_veri.py          # sentetik perakende verisi (6511 satış)

# 5) Soru sor (etkileşimli)
.venv/bin/python sor.py
#   → "en çok ciro yapan 5 ürün hangisi?" yaz, Enter
```

Üretimde dikkat

- Uygulama, **salt-okunur** bir DB kullanıcısı kullanmalı (SORGU_DATABASE_URL). Yazma yetkisi yalnızca veri kurulumunda.
- 8GB makinede: launchctl setenv OLLAMA_MAX_LOADED_MODELS 1 ile aynı anda tek model yüklenir (çökme önlenir).
- Gerçek şirket verisi **kullanma** — demo sentetiktir; kendi şemanı bağlarken .env ve read-only kullanıcı kur.

12. Doğruluk Kanıtı ve Test

Proje “çalışıyor” demekle yetinmez; **ölçer**. Dört bağımsız test eksenini:

Test	Ne ölçer	Sonuç
test_guvenlik.py	Yazma/injection reddi (K1)	15/15
sql_kontrol grounding	Uydurma kolon/tablo reddi (K2)	10/10
gold_set.py	Uçtan uca execution accuracy	7/8 (%87.5)
metrik_test.py	Türetilmiş metrik doğruluğu (K3)	3/3

Kontrol katmanlarının kanıtı

K3 olmasaydı sistem “Bal, toplam ciro içinde %25.1” derdi (yanlış — sadece 5 satırın toplamı). K3 sayesinde **doğru** cevap: “%10.7” (gerçek toplam 1.646.768 ayrı çekilerek). Bu fark, mühendisliğin doğruluğa katkısının somut kanıtıdır.

13. Yol Haritası

Faz 3 — Doğruluk derinliği

- Koşullu **LLM-critic** (clause-bazlı anlam denetimi)
- **Güven skoru** (iki-aday uyumsuzluğu → “emin değilim”)
- **Intent-based retrieval** (Pinterest): geçmiş SQL’leri doğal dile çevirip RAG’e koy
- **Kolon budama** (Uber): büyük şemada context daraltma

Faz 4 — Ölçek & vitrin

- **Semantik metrik katmanı** (ciro, aktif kullanıcı... sabit, onaylı tanımlar)
- Kiralık GPU’da **32B** ile ileri analitik
- README + mimari diyagram + demo GIF
- Basit web arayüzü (tek sayfa)

14. Kapanış

Bu proje, kurumsal text-to-SQL'in zor ve çözülmemiş bir problem olduğunu kabul ederek, ona **mühendislik disipliniyle** ve **veriyi hiç dışarı çıkarmadan** yaklaşır. Doğruluğu modelden değil; şema linkleme, RAG few-shot, çok-katmanlı kontrol, türetilmiş-metrik denetimi ve halüsinasyonsuz rapor **mimarisinden** alır.

Sonuç: 1.9 GB'lık bir model, bir dizüstünde, gündelik iş sorularını %87–95 doğrulukla, **dürüstçe ölçülmüş** ve **kontrol altına alınmış** biçimde yanıtıyor — sınırlarını da açıkça belgeleyerek.

Daha fazlası

Kaynak kod: [‹GitHub-linki›](#) · **Kurulum & demo videosu:** [‹video-linki›](#) · **İletişim / LinkedIn:** [‹profil-linki›](#)

Bu doküman Typst ile üretildi. Kaynak: `docs/sirket_sorgu_tanitim.typ` — metni güncelleyip `typst compile` ile yeniden üretebilirsiniz.